

The echo statement mentioned in the code here is used to show that the commands in .bashrc file are executed on bootup as well as when connecting to bash.

The third method to run a program on Raspberry Pi is by using the init.d directory. As with the previous steps, add the file with the code to the /etc/init.d directory. This directory contains that are initiated on bootup. Add the lines to the sample script and convert it to a Linux Standard Base(LSB).

```
• # /etc/init.d/sample.py
• ### BEGIN INIT INFO
• # Provides:          sample.py
• # Required-Start:    $remote_fs $syslog
• # Required-Stop:     $remote_fs $syslog
• # Default-Start:     2 3 4 5
• # Default-Stop:      0 1 6
• # Short-Description: Start daemon at boot time
• # Description:       Enable service provided by daemon.
• ### END INIT INFO
```

Run the command finally in the command-line by typing `sudo update-rc.d sample.py defaults`.

An important step in the DIY process is to know how to create a unit file that can be executed with a device much later on if necessary. First, open a sample file using the command below:-

```
• sudo nano /lib/systemd/system/sample.service
```

Add the following texts:-

```
• Description=My Sample Service
• After=multi-user.target
• Type=idle
• ExecStart=/usr/bin/python /home/pi/sample.py
• WantedBy=multi-user.target
```

Save and exit the editor. Move on to the configure tab and configure the system. A new service will be initialised in the launch environment on setup. The “ExecStart” element helps in specifying the command that is to be run. In this case, type refers to the status of the command which is set to idle. Note that the paths are absolute and define the complete location of Python as well as the location of the script. Change the line to:-

```
• ExecStart=/usr/bin/python /home/pi/sample.py > /home/pi/sample.log 2>&1
• sudo chmod 644 /lib/systemd/system/sample.service
```